



VPPaaS - Sprint 2 Report

Virtual Power Plant-as-a-Service

Pedro Pereira (98962)

João Guerreiro (1116375)

June 7, 2026

Contents

1	Introduction	3
1.1	Documentation	3
1.2	Architecture Changes	3
1.3	Terraform Changes	4
2	Information Flows and Business Processes	5
2.1	Prosumer Management Business Process	5
2.2	Utility Operator Management Business Process	7
2.3	Asset Link Management Business Process	9
2.4	Telemetry Ingestion Business Process	11
2.5	Flexibility Emission Business Process	12
2.6	Grid Balancing Recommendation Business Process	12
2.7	Energy Analytics Business Process	14
2.8	Flexibility Forecasting Business Process	15
3	Implementation and Deployment	16
3.1	Kong	16
3.2	Camunda	16
3.3	Deployment Automation	17
4	Test	17
4.1	Kafka	17
4.2	Microservices and Kong	17
4.3	Camunda	18
4.4	End-to-End Tests	19
4.5	E2E Testing Conclusions	22

1 Introduction

In Sprint 2, the VPPaaS project evolved from the Sprint 1 microservice baseline into a complete integrated platform with business process orchestration, an API gateway, cloud deployment automation and end-to-end validation. Sprint 1 delivered the main Quarkus microservices, AWS RDS persistence, Kafka messaging, Terraform deployment and service-level tests. Sprint 2 added Camunda 8 BPMN processes, Kong API Gateway, a reorganised two-account AWS deployment, AMI-based container startup, and automated tests that validate the path from BPMN execution to the underlying database.

The platform supports the core operations of a Virtual Power Plant-as-a-Service system: prosumer management, utility operator and grid-zone management, asset link management, telemetry ingestion, flexibility event emission, grid balancing recommendation, energy analytics and AI-assisted flexibility forecasting. The final architecture combines REST communication through Kong, asynchronous integration through Kafka, persistence in AWS RDS, and process execution through Camunda.

The main additions of Sprint 2 were:

- Camunda 8 BPMN process definitions for all required VPPaaS business operations;
- Kong API Gateway exposing all eight microservices through one public entry point;
- deployment reorganisation into two AWS Academy accounts due to EC2 limits;
- one EC2 host per microservice/service role, with FlexibilityForecasting co-located with Ollama;
- pre-baked Docker AMIs created by `CreateAMI.sh`, avoiding Docker installation during each deployment;
- updated lifecycle scripts: `Build.sh`, `CreateAMI.sh`, `Deploy.sh`, `Undeploy.sh` and `clean-all-projects.sh`;
- corrected GridBalancing algorithm using battery, solar and EV charger telemetry, per-zone safety thresholds and neighbouring-zone logic;
- JUnit/Quarkus tests, shell integration tests and a BPMN end-to-end test.

1.1 Documentation

The project documentation and repository structure were updated to describe the Sprint 2 architecture and deployment flow. The root `README.md` explains the operational scripts, credential files, deployment order, Kong routes, BPMN processes and test sequence. The `docs/` folder contains supporting documentation and BPMN process specifications.

The main source folders are now:

- `microservices/`: source code for the eight Quarkus microservices;
- `BPMN/`: executable BPMN process definitions;
- `Sequence-Diagrams/`: PlantUML sequence diagrams for the business flows;
- `terraform/Account1/`: Account 1 infrastructure and core microservices;
- `terraform/Account2/`: Account 2 dependent services and auxiliary components;
- `tests/`: shell-based integration tests, address discovery and the BPMN end-to-end test.

The main operational scripts are located at the repository root. `Build.sh` builds and publishes the Docker images, `CreateAMI.sh` prepares Docker-ready AMIs for both AWS accounts, `Deploy.sh` provisions the complete system and configures Kong routes, and `Undeploy.sh` destroys the infrastructure in reverse dependency order.

1.2 Architecture Changes

The main change from Sprint 1 was architectural: the project stopped being only a set of independent microservices and became an integrated platform with orchestration, gateway access and a clearer cloud deployment topology.

The application layer was kept as eight Quarkus microservices, but their role in the system changed. Instead of being tested mostly as isolated services, they are now called by BPMN processes through a shared gateway entry point. This means that Camunda can execute the business flows without depending directly on the changing DNS address of each EC2 instance.

The cloud architecture was also reorganised. The deployment was split across two AWS Academy accounts to respect the EC2 limits, and the microservices were separated into their own EC2 instances instead of being grouped as before. Account 1 contains the shared infrastructure and the core services, while Account 2 contains the dependent services and the AI/forecasting components.

Some service responsibilities were adjusted to fit this architecture. EnergyAnalytics now publishes analytical outputs that can be consumed by other services, GridBalancing was updated to use richer telemetry and grid-zone data, and FlexibilityForecasting was placed together with Ollama so the AI call can be made locally on the same EC2 instance.

Overall, Sprint 2 changed the system architecture from a microservice prototype into a deployed end-to-end platform composed of BPMN orchestration, gateway routing, Kafka events, RDS persistence and independent cloud-hosted services.

1.3 Terraform Changes

The infrastructure was reorganised into two AWS Academy accounts. This change was required because each Academy account has a limit on the number of EC2 instances. The new structure deploys the core infrastructure and independent services in Account 1, while dependent services and auxiliary components run in Account 2.

Account	Instances	Services
Account 1	8 EC2 + 1 RDS	Kafka 1, Kafka 2, Kafka 3, Kong, Camunda, Prosumer, UtilityOperator and Telemetry
Account 2	6 EC2	Konga, Ollama/FlexibilityForecasting, AssetLink, FlexibilityEvent, EnergyAnalytics and GridBalancing

Table 1: Sprint 2 AWS Academy deployment split.

The active Terraform tree is:

```

terraform/
Account1/
  RDS/
  Kafka/
  Kong/
  Camunda/
  Prosumer/
  UtilityOperator/
  Telemetry/
Account2/
  Konga/
  Ollama/
  AssetLink/
  FlexibilityEvent/
  EnergyAnalytics/
  GridBalancing/

```

Each microservice EC2 runs one Docker container on port 8080. Startup scripts pull the corresponding Docker Hub image and inject runtime configuration through environment variables,

including the RDS endpoint, Kafka brokers and upstream service URLs. The exception is the Ollama instance: Ollama runs natively on port 11434, while FlexibilityForecasting runs as a Docker container on the same host using the host network so it can reach Ollama locally.

A new no-rebuild deployment strategy was added. `CreateAMI.sh` creates Docker-ready AMI snapshots for Account 1 and Account 2. These AMI IDs are saved in `amis.env`, which is gitignored and later consumed by `Deploy.sh`. This avoids repeatedly installing Docker on every EC2 instance during deployment.

Secrets were also separated from Terraform files. AWS temporary credentials are loaded through `access.sh` and `access2.sh`; Docker Hub credentials are passed as Terraform variables; SSH keys and generated AMI/address files are gitignored. The `terraform.tfvars` files therefore contain infrastructure values rather than secrets.

2 Information Flows and Business Processes

Sprint 2 turns the VPPaaS information flows into executable BPMN processes. The BPMN files are stored in `BPMN/`. Service tasks call the microservices through Kong, while user tasks are used where a human approval or review step is required. In the Camunda 8 processes, service tasks use the REST outbound connector `io.camunda:http-json:1`; the HTTP method and URL are configured through Zeebe input mappings. BPMN URLs use a placeholder for the Kong address, which is replaced during deployment or testing.

2.1 Prosumer Management Business Process

The Prosumer Management process handles onboarding, updating and removal of prosumers. It is structured as a hub-and-spoke design: the **ProsumerHub** process presents a user task where the operator selects an operation (create, update or delete) and then delegates to the corresponding sub-process via a Call Activity.

The **Creation** sub-process (`ProsumerCreationInitiator` / `ProsumerCreationExecutor`) includes a KYC/business validation user task before calling `POST /Prosumer` through Kong.

The **Deletion** sub-process (`ProsumerDeletionInitiator` / `ProsumerDeletionExecutor`) fetches the target prosumer, asks the manager to confirm the operation, and calls `DELETE /Prosumer/{id}` only after explicit approval. A message correlation protocol with four message types (`Msg_Del_Decline`, `Msg_Del_Declare`, `Msg_Del_Accept`, `Msg_Del_Reject`) ensures the initiator and executor remain synchronised.

The **Update** sub-process (`ProsumerUpdateInitiator` / `ProsumerUpdateExecutor`) follows the same two-pool collaboration pattern as Deletion. The initiator collects new prosumer data (`name`, `fiscalnumber`, `location`), the executor reviews the current record and calls `PUT /Prosumer/{id}/{name}/{fiscalnumber}/{location}` on approval.

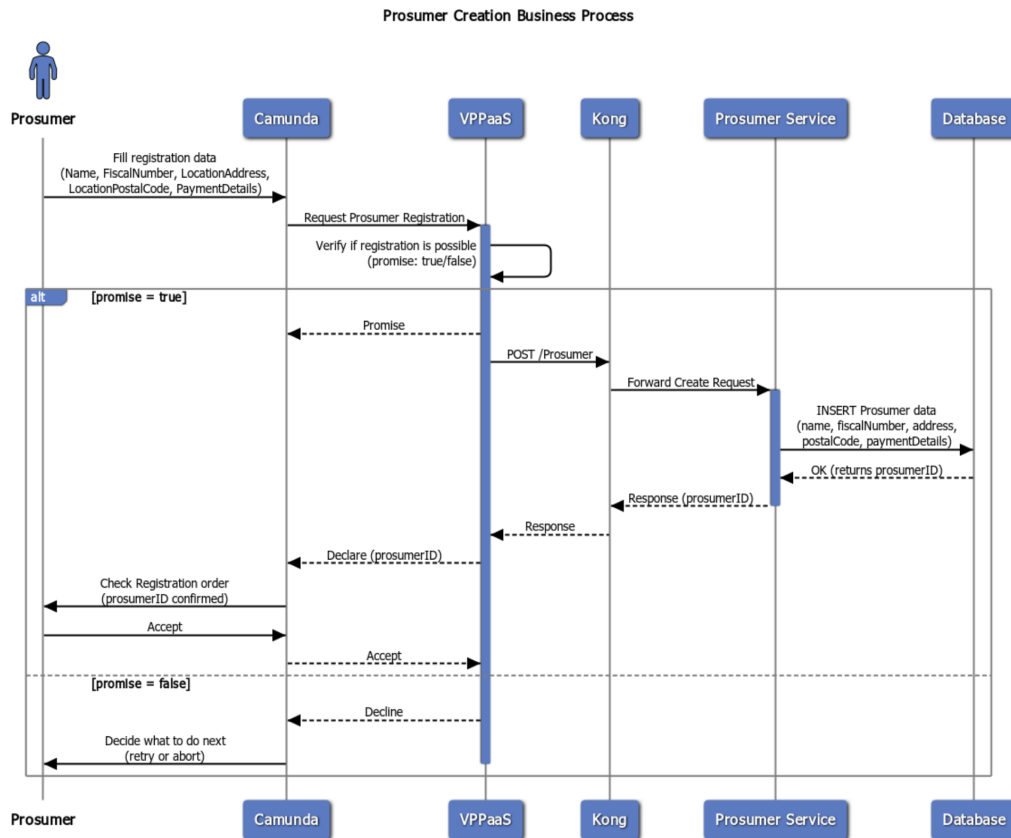


Figure 1: Sequence Diagram: Prosumer Creation Business Process.

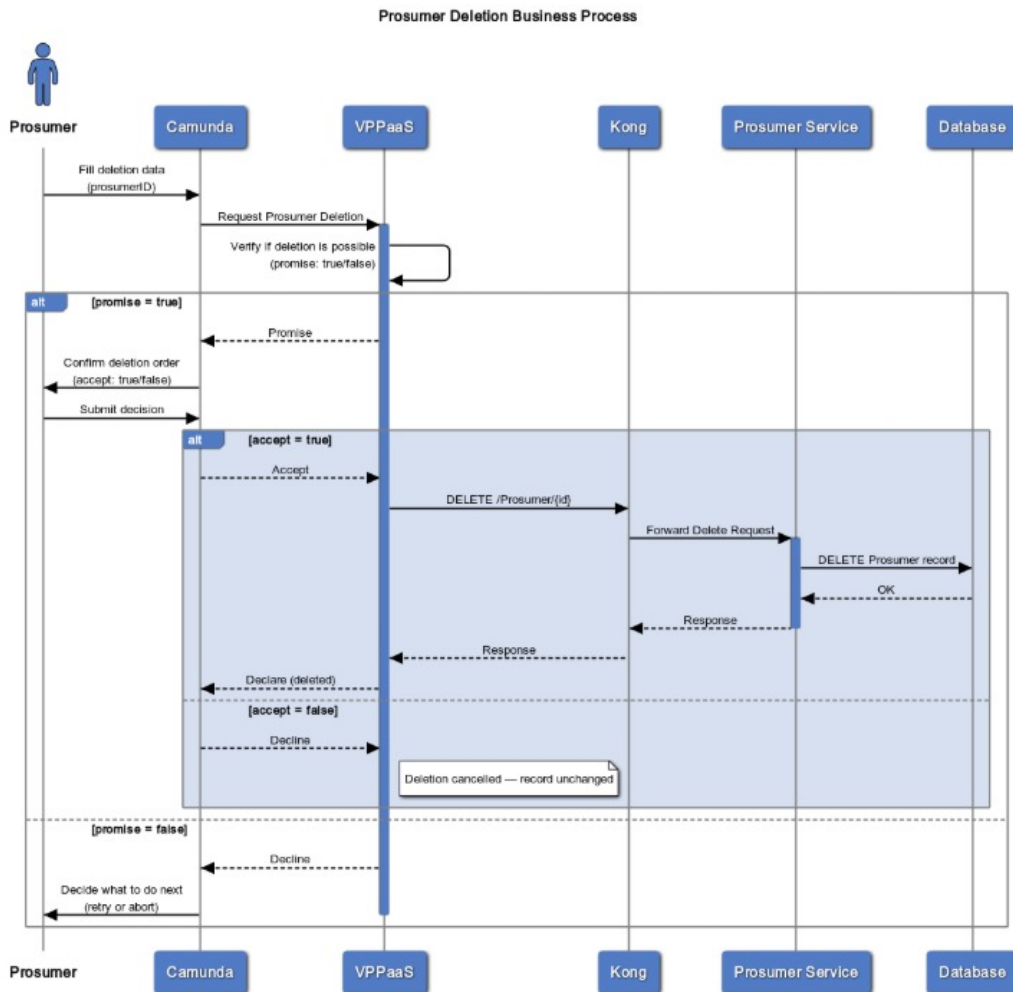


Figure 2: Sequence Diagram: Prosumer Deletion Business Process.

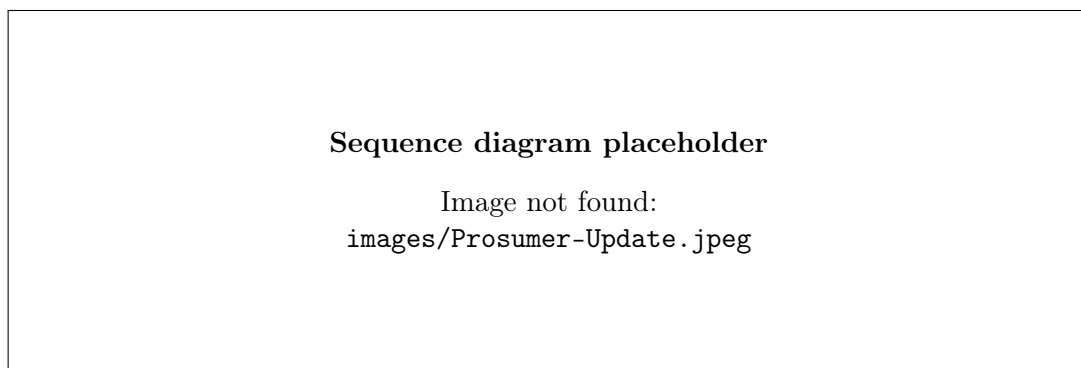


Figure 3: Sequence Diagram: Prosumer Update Business Process.

2.2 Utility Operator Management Business Process

The Utility Operator Management process creates and deletes utility operators and supports grid-zone management. Utility operators are exposed through `/UtilityOperator`; grid zones are exposed through `/GridZone`. Grid zones provide the capacity, safety threshold and postal code information used later by the balancing algorithm.

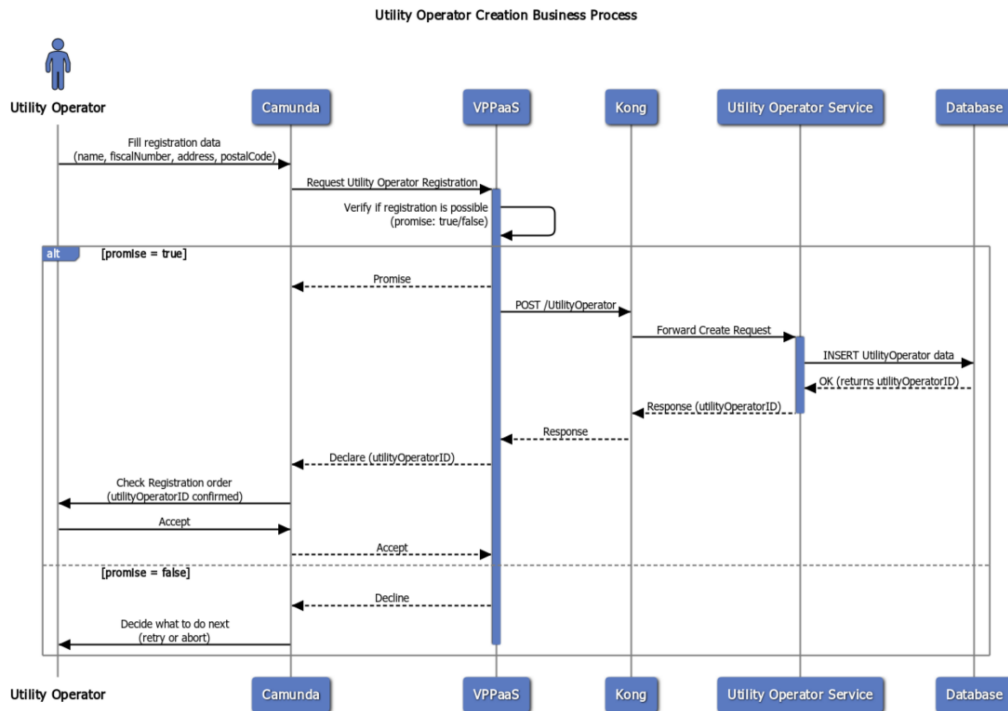


Figure 4: Sequence Diagram: Utility Operator Creation Business Process.

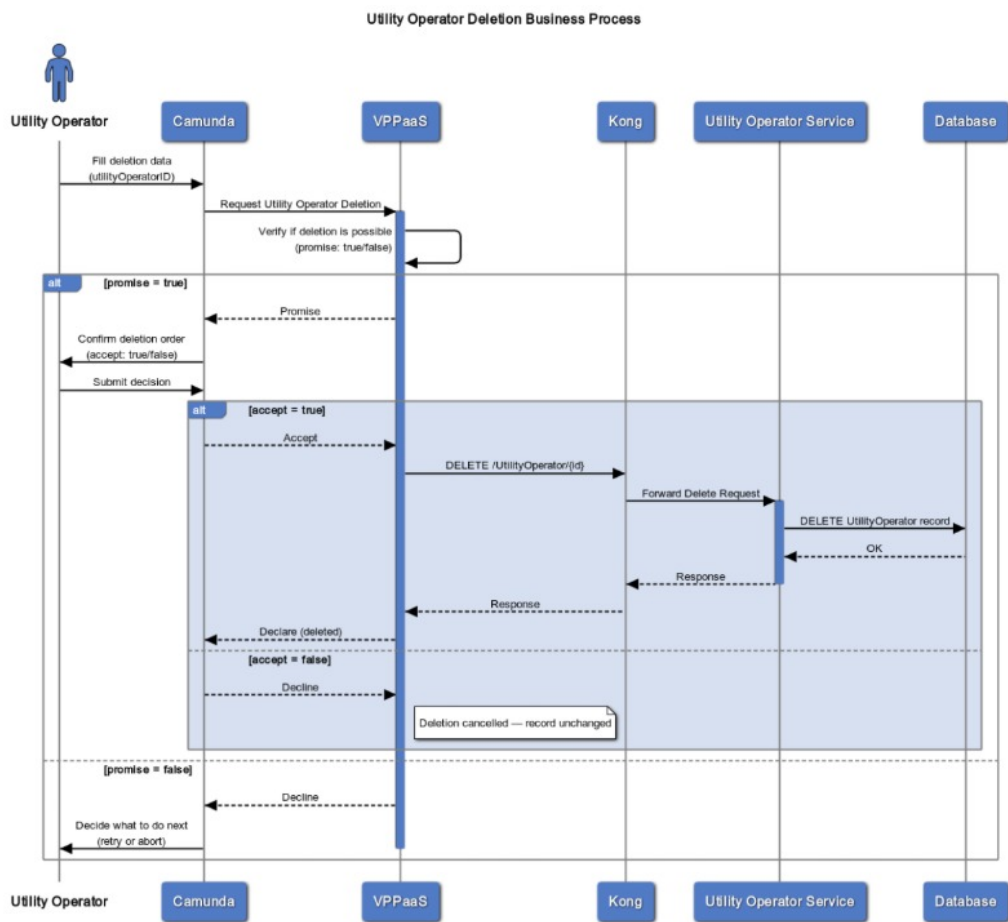


Figure 5: Sequence Diagram: Utility Operator Deletion Business Process.

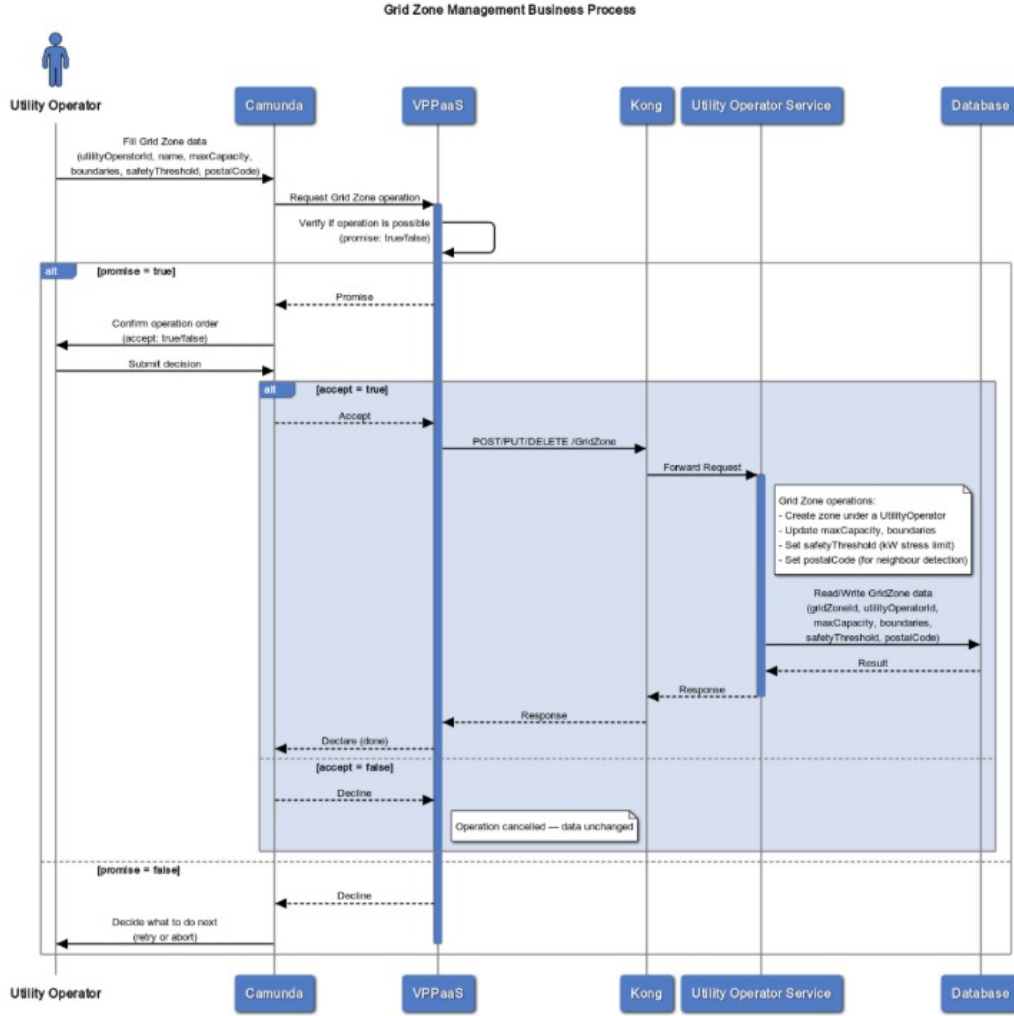


Figure 6: Sequence Diagram: Grid Zone Management Business Process.

2.3 Asset Link Management Business Process

The Asset Link process establishes or revokes the digital contract between a prosumer and a utility operator. Asset links define which assets are available for monitoring and analytics. When an asset link is created, the AssetLink service also creates or identifies the Kafka topic associated with that relationship and registers telemetry consumption.

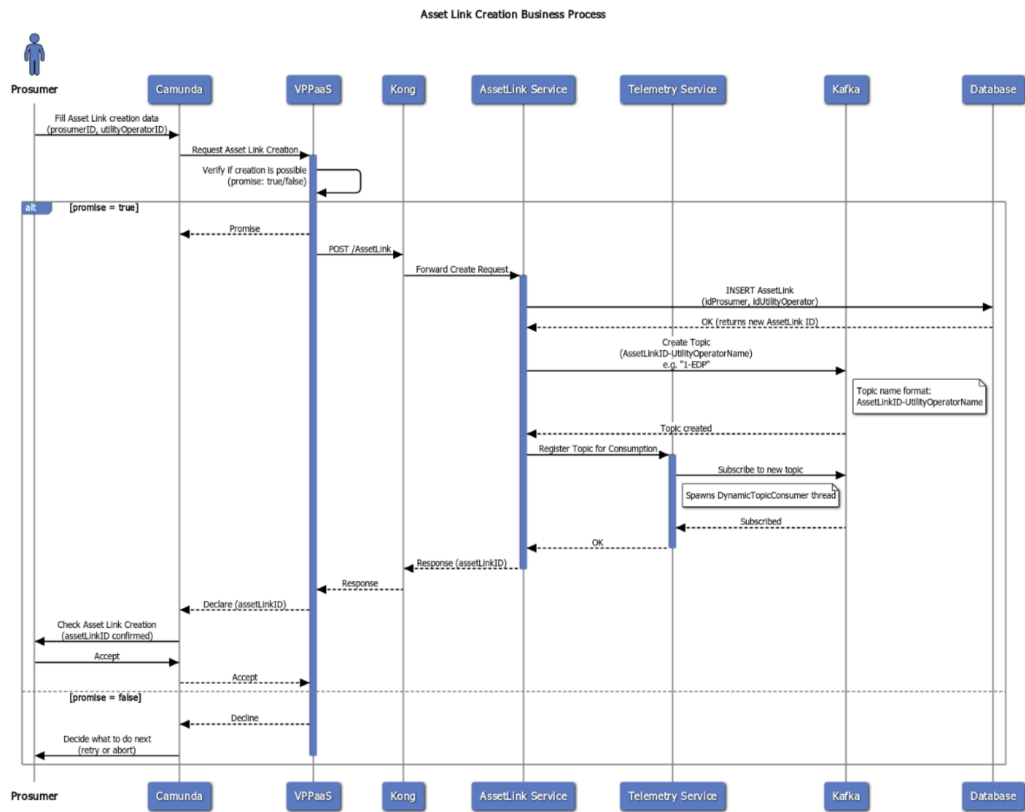


Figure 7: Sequence Diagram: Asset Link Creation Business Process.

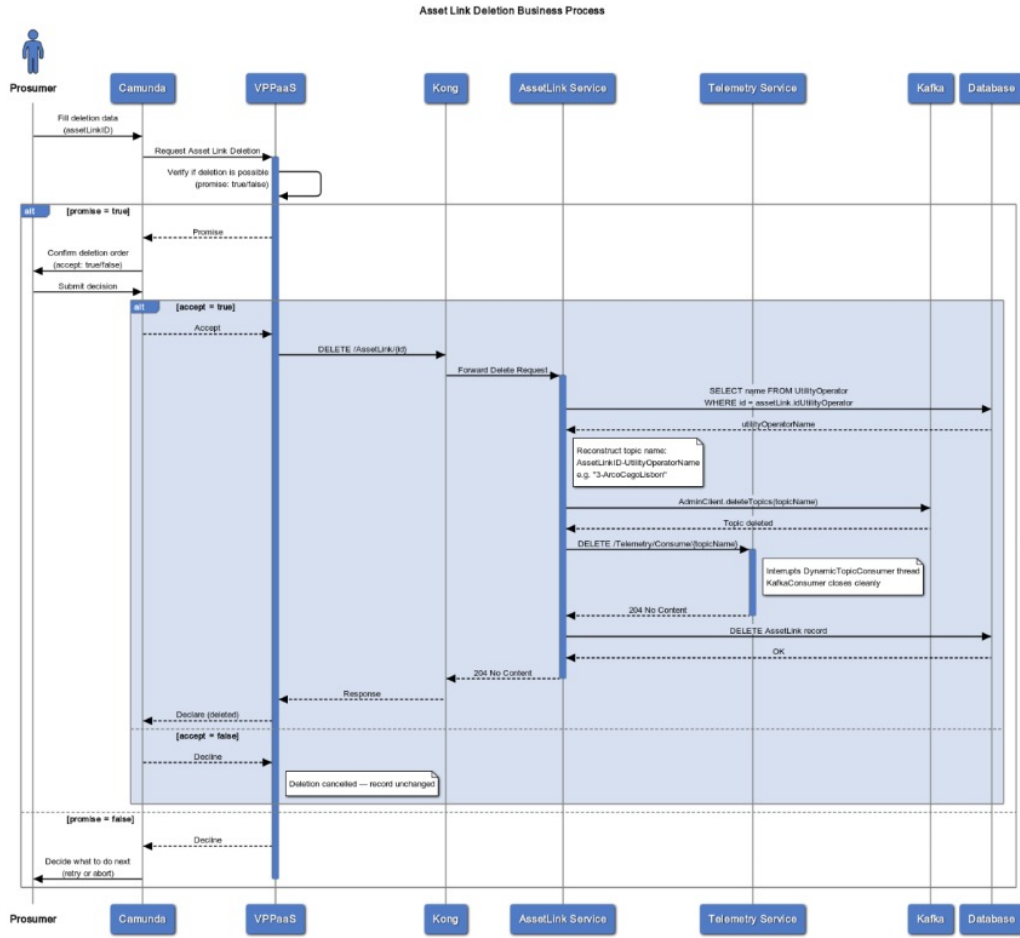


Figure 8: Sequence Diagram: Asset Link Deletion Business Process.

2.4 Telemetry Ingestion Business Process

Telemetry Ingestion registers a Kafka consumer and stores simulator readings in the database. The Telemetry microservice supports `POST /Telemetry/Consume` to register a consumer for a topic and `GET /Telemetry` to retrieve stored readings. Telemetry messages represent battery, solar and EV charger assets.

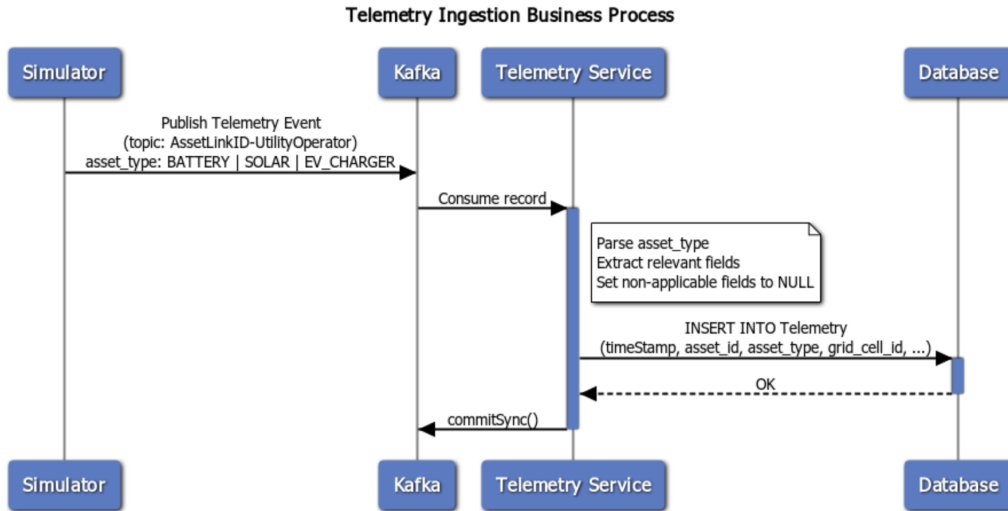


Figure 9: Sequence Diagram: Telemetry Ingestion Business Process.

2.5 Flexibility Emission Business Process

The Flexibility Emission process evaluates telemetry and creates flexibility events when operational opportunities are detected. These events are persisted and published to Kafka, allowing other services or operators to consume the resulting flexibility information.

The process calls `POST /FlexibilityEvent/trigger`.

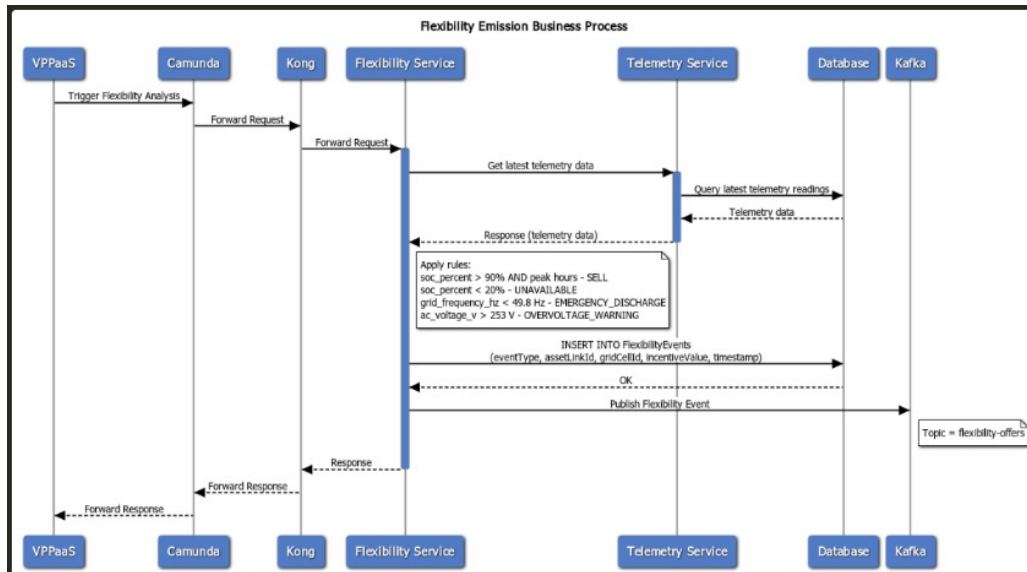


Figure 10: Sequence Diagram: Flexibility Emission Business Process.

2.6 Grid Balancing Recommendation Business Process

The Grid Balancing Recommendation process requests balancing recommendations between grid zones. The service computes net load using EV charging, battery discharge and solar generation. It compares each zone with its `safetyThreshold`, searches only for neighbouring candidate zones when postal codes are available, stores the recommendation in RDS and publishes it to the Kafka topic `grid-balancing-recommendation`.

Additionally, the GridBalancing microservice includes a system to accept and act upon a generated recommendation. When an operator accepts a recommendation, the system executes

the necessary changes (such as moving prosumers to the deficit zone and creating new Asset Links) to perform the grid balancing operation.

The recommendation generation process calls POST /GridBalancing/recommend, and the acceptance flow calls POST /GridBalancing/act/{recommendationId}.

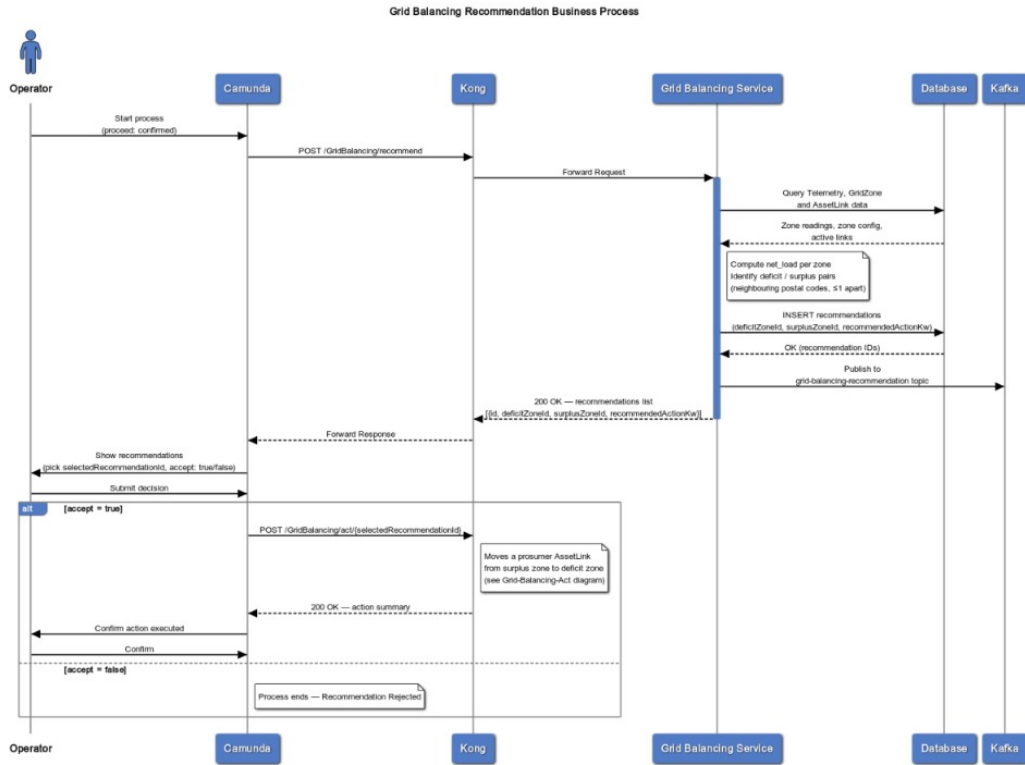


Figure 11: Sequence Diagram: Grid Balancing Recommendation Business Process.

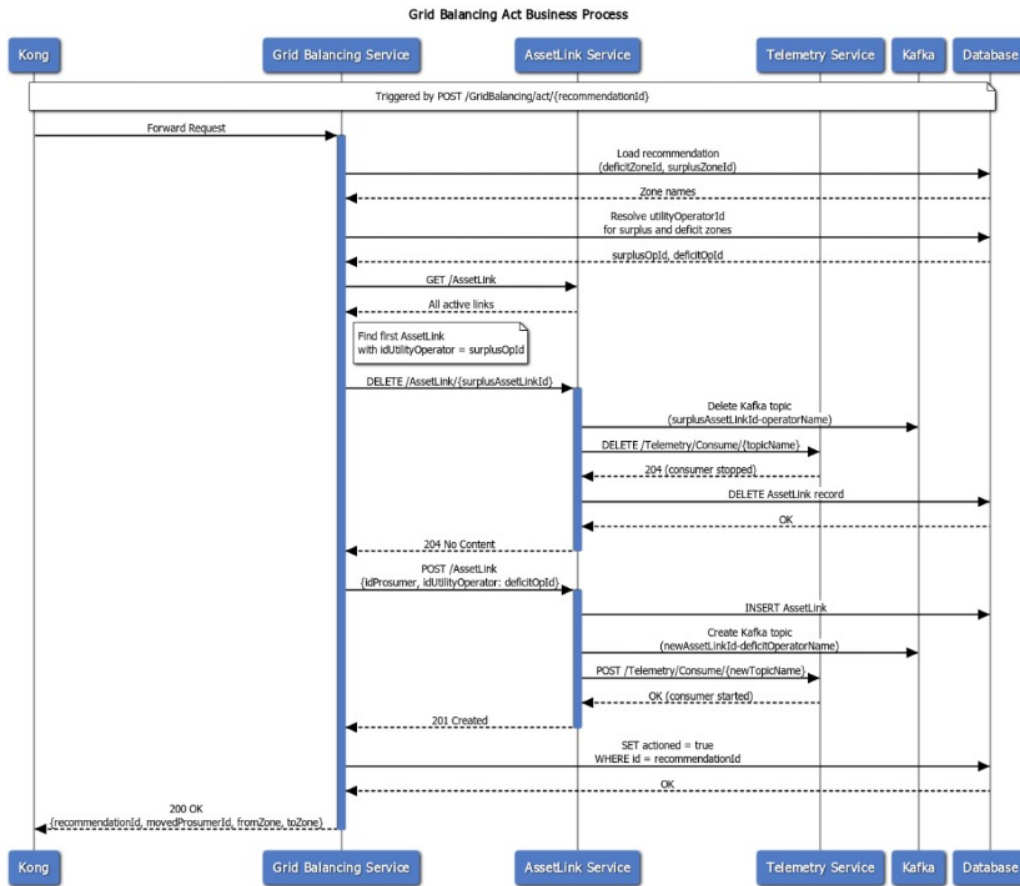


Figure 12: Sequence Diagram: Grid Balancing Act Business Process.

2.7 Energy Analytics Business Process

The Energy Analytics process computes aggregated indicators from telemetry. It produces energy discharged by zone, generated energy by prosumer, consumed energy by prosumer and average state of charge. The outputs are persisted and published to Kafka so they can be consumed by downstream processes, including GridBalancing.

The process calls `POST /EnergyAnalytics/compute`.

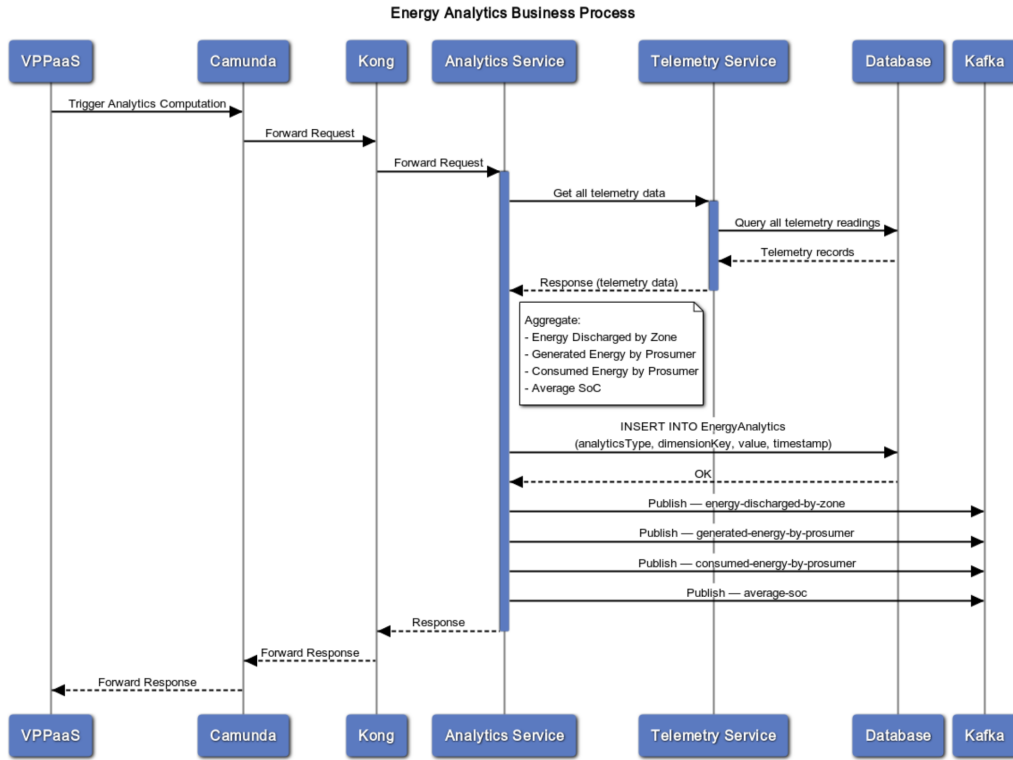


Figure 13: Sequence Diagram: Energy Analytics Business Process.

2.8 Flexibility Forecasting Business Process

The Flexibility Forecasting process uses the AI component of the system. It sends recent flexibility-event context to the FlexibilityForecasting microservice, which calls Ollama/Llama 3.2 and returns an AI-assisted forecast or analysis. This supports operational decision-making based on previous flexibility events.

The process calls `POST /FlexibilityForecasting/forecast`.

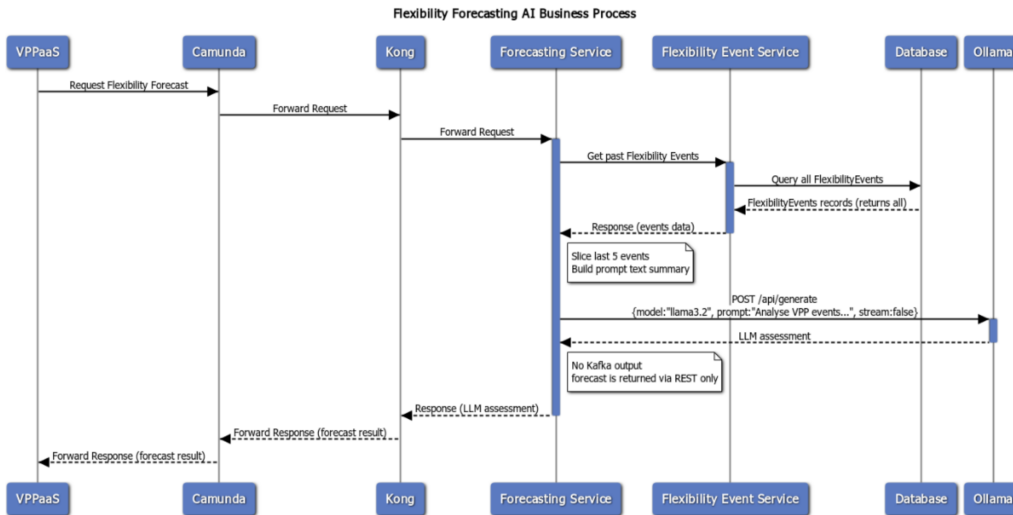


Figure 14: Sequence Diagram: Flexibility Forecasting AI Business Process.

3 Implementation and Deployment

This section summarises the main implementation and deployment decisions introduced in Sprint 2.

3.1 Kong

Kong was updated to work as the shared entry point for the Sprint 2 platform. The main change was to configure one route for each microservice, allowing Camunda and the test scripts to access the services through a single gateway address instead of calling each EC2 instance directly.

The deployment script now configures the required routes after both AWS accounts are deployed. The relevant change is that the gateway routing was aligned with the current microservice deployment and with the BPMN processes, so requests can flow consistently from Camunda to the backend services.

3.2 Camunda

Camunda is deployed in Account 1 using C8Run 8.8.9. This runtime provides Zeebe, Operate, Tasklist, Elasticsearch and the Connectors runtime in a single simplified deployment, which is appropriate for the project and the AWS Academy environment. All web applications and the REST API are served through port 8080; the development login is `demo/demo`.

BPMN service tasks use the REST outbound connector `io.camunda:http-json:1`. User tasks that need to be completed through the API include the `<zeebe:userTask />` extension, making them searchable and completable through the Zeebe REST API v2.

BPMN files are deployed with the Camunda REST API:

```
curl -u demo:demo -X POST http://<camunda-host>:8080/v2/deployments \
-F "resources=@process.bpmn"
```

Before deployment, the Kong placeholder is replaced with the live Kong DNS name. This keeps BPMN files environment-independent and avoids committing cloud-specific addresses.

3.3 Deployment Automation

The Sprint 2 lifecycle is composed of four main steps:

1. **Build.sh**: builds all Quarkus microservices with the production profile and pushes Docker images to Docker Hub;
2. **CreateAMI.sh**: creates Docker-ready AMI snapshots for Account 1 and Account 2 and saves the IDs in **amis.env**;
3. **Deploy.sh**: deploys Account 1, writes **account1-addresses.env**, switches to Account 2, deploys dependent services, configures Kong routes and prints the final endpoints;
4. **Undeploy.sh**: destroys Account 2 first and then Account 1 in reverse dependency order.

The full lifecycle is:

```
./CreateAMI.sh  
./Deploy.sh  
./Undeploy.sh
```

In Account 1, **Deploy.sh** applies RDS, Kafka, Kong, Camunda, Prosumer, UtilityOperator and Telemetry. It then writes **account1-addresses.env** with the live endpoints. In Account 2, it deploys Konga, AssetLink, FlexibilityEvent, Ollama/FlexibilityForecasting, EnergyAnalytics and GridBalancing. Runtime dependencies such as RDS address, Kafka brokers and upstream service URLs are injected into the corresponding **terraform.tfvars** files during deployment.

4 Test

This section describes how the project was validated after the Sprint 2 integration work.

4.1 Kafka

Kafka testing validates the event-driven part of the platform. AssetLink creates or identifies Kafka topics for active asset links. The VPPaaS simulator publishes telemetry messages to those topics, and the Telemetry service consumes them through dynamically registered consumers and stores the readings in RDS.

The project uses a three-broker Kafka cluster. Telemetry consumers are registered through **POST /Telemetry/Consume**. The simulator then generates battery, solar and EV charger readings, allowing downstream services such as FlexibilityEvent, EnergyAnalytics and GridBalancing to operate on real telemetry data.

4.2 Microservices and Kong

All microservices now have JUnit 5 / QuarkusTest classes. The test set covers CRUD operations, trigger endpoints and compute/recommend/forecast endpoints. The classes include **ProsumerResourceTest**, **UtilityOperatorResourceTest**, **TelemetryResourceTest**, **TelemetrySimulatorTest**, **AssetLinkResourceTest**, **FlexibilityEventResourceTest**, **EnergyAnalyticsResourceTest**, **GridBalancingResourceTest** and **FlexibilityForecastingResourceTest**.

The **tests/** folder contains shell scripts for deployed integration tests:

```
prosumer.sh
utilityoperator.sh
gridzone.sh
assetlink.sh
telemetry.sh
flexibilityevent.sh
energyanalytics.sh
gridbalancing.sh
forecast.sh
bpmn_test.sh
get-addresses.sh
```

The recommended service-level order is:

```
bash tests/prosumer.sh
bash tests/utilityoperator.sh
bash tests/gridzone.sh
bash tests/assetlink.sh
bash tests/telemetry.sh
bash tests/flexibilityevent.sh
bash tests/energyanalytics.sh
bash tests/gridbalancing.sh
bash tests/forecast.sh
```

This order respects data dependencies: base entities are created first, asset links define telemetry topics, telemetry feeds analytical services, and forecasting depends on flexibility-event information. Kong is validated by calling the public gateway routes and confirming that the requests reach the correct backend service with `strip_path=false`.

The root script `run_all_tests.sh` automates the test sequence. It loads live addresses, starts the telemetry simulator, waits for ingestion, executes the dependent service tests and prints a pass/fail/skip summary. The Forecast test is skipped when the `llama3.2` model is not yet loaded in Ollama, and the script prints the pull command needed to load it.

4.3 Camunda

The BPMN end-to-end test is implemented in `tests/bpmn_test.sh`. It validates the complete Sprint 2 orchestration path:

Camunda BPMN → Kong → Quarkus microservice → RDS

The test performs the following steps:

1. resolves Camunda and Kong addresses from Terraform state or command-line arguments;
2. deploys the BPMN files after replacing the Kong placeholder;
3. starts a `ProsumerCreationInitiator` process instance with test variables;
4. waits for the KYC user task through `/v2/user-tasks/search`;
5. completes the task with `approved=true`;
6. waits for the service task to call `POST /Prosumer` through Kong;
7. verifies through the service layer that the prosumer was created in the database.

During testing, several Camunda-specific details were identified: deployments use `/v2/deployments`, process instances use `/v2/process-instances`, user tasks are completed through `/v2/user-tasks/{key}/complete` and `processDefinitionKey` must be sent as a JSON string because Camunda keys are 64-bit values.

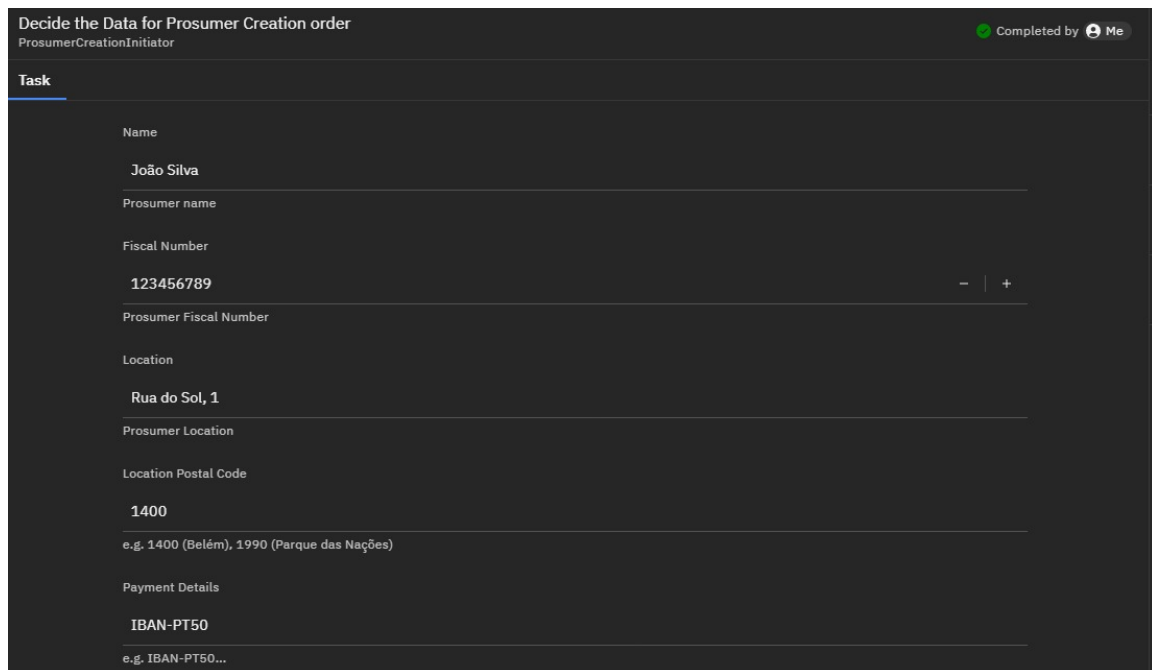
4.4 End-to-End Tests

The End-to-End tests were documented with execution evidence instead of using a dedicated testing automation framework. This follows the course guidance: the objective is not to demonstrate knowledge of a specific testing framework, but to show that the complete integrated system was executed and that the expected results were obtained.

The tested scenario follows the main VPPaaS business path:

1. create or verify the Prosumer that owns the energy assets;
2. create or verify the Utility Operator responsible for the grid area;
3. create grid zones associated with the utility operator;
4. create an Asset Link between the prosumer and the utility operator;
5. inject telemetry through the deployed API so that one grid zone has surplus energy;
6. run the GridBalancing recommendation flow;
7. verify that the generated recommendation is available and can be selected/accepted by the operator.

This validates the system as an integrated platform because the flow depends on the UI/API state, Kong routing, microservice communication, telemetry data, the GridBalancing recommendation logic and persisted data.



The screenshot shows a web form titled "Decide the Data for Prosumer Creation order" with the subtitle "ProsumerCreationInitiator". In the top right corner, there is a green status indicator and the text "Completed by Me". Below the title, there is a "Task" tab. The form contains several input fields with labels and values:

- Name:** João Silva
- Prosumer name:** (empty field)
- Fiscal Number:** 123456789
- Prosumer Fiscal Number:** (empty field)
- Location:** Rua do Sol, 1
- Prosumer Location:** (empty field)
- Location Postal Code:** 1400
- Payment Details:** IBAN-PT50

Below the "Payment Details" field, there is a hint text: "e.g. IBAN-PT50...".

Figure 15: End-to-End evidence: Prosumer was created and is visible in the Prosumer hub.

Request Utility Operator Creation
UtilityOperatorManagement
Assigned to me
Unassign

Task
Process

Name

SolarGrid SA

Utility Operator name

Location

Lisbon

Utility Operator Location

Figure 16: End-to-End evidence: Utility Operator creation/selection in the operator hub.

Request Grid Zone Creation
GridZoneCreation
Assigned to me
Unassign

Task
Process

Utility Operator ID*

5

ID of the utility operator that owns this zone

Zone Name*

Belém

e.g. Belém, Parque das Nações

Max Capacity (kW)*

500

Maximum energy capacity in kilowatts

Safety Threshold (kW)*

400

Load level (kW) above which the zone is considered stressed

Postal Code*

1400

Used to detect neighbouring zones for grid balancing

Figure 17: End-to-End evidence: first Grid Zone created for the balancing scenario.

Figure 18: End-to-End evidence: second Grid Zone created, allowing the balancing test to compare zones.

Figure 19: End-to-End evidence: Asset Link created between the Prosumer and the Utility Operator.

After confirming that the entities were created, the next step was to generate telemetry that creates surplus in one grid zone. For this, telemetry was sent through Kong using a payload with high solar generation and low EV load. This creates a zone with local surplus, which can then be used by the GridBalancing flow as a candidate surplus zone.

```
curl -X POST http://$KONG:8000/Telemetry \
-H "Content-Type: application/json" \
-d '{
  "assetLinkId": 1,
  "solarGeneration": 250.0,
  "batteryDischarge": 0.0,
  "evChargerLoad": 10.0,
  "timestamp": "<current UTC timestamp>"
}'
```

In the executed test, the created entities were checked in the application screens and selected from the available dropdowns. This was important to confirm that the data was not only

stored in the backend, but also visible to the user-facing flow. Once the surplus telemetry existed, the recommendation generation was executed and the operator could select the generated recommendation and accept it.

The expected result of this End-to-End test is that the system can start from created business entities, receive telemetry, identify a surplus grid zone, generate a balancing recommendation and allow the operator to accept the proposed recommendation. This demonstrates the complete Sprint 2 integration from data creation to operational decision support.

To automate and demonstrate this entire flow programmatically, we have also provided an executable test script located at `tests/e2e-gridbalancing.sh`. This script simulates the complete End-to-End process described above.

4.5 E2E Testing Conclusions

There was an issue regarding the AWS Academy credentials that didn't allowed us to showcase the full project working. We discovered a bug on the gridbalancing component that blocked us from testing further. The rest of the components are working as intended including the Ollama service, creation and deletion of components and also any other service. Having also the tests and JUnit unit tests allowed us to be confident on the data and further testing was done before the implementation of this section assuring completion on the course requirements.